Research & Evaluation Report: Toy Blockchain & Ledger Simulator

1. Required Investigations

1.1 Tamper-Evidence & Integrity Constraints

To evaluate the cryptographic defense capabilities of the architectural ledger design, an adversarial attack simulation was performed. A transaction value within a confirmed historical block (Block 1) was manually altered from its original honest value to a fraudulent amount.
Upon running the full-chain validation execution loop following the data modification, the engine instantly identified the security breach and flagged the ledger state as invalid:

- **Detection Mechanism:** The compromise was detected by the Data Hash Recomputation Check.
- **Technical Analysis:** A block's hash is generated over a stable, deterministic serialization of its core internal state properties: Index, Timestamp, Transactions, PrevHash, and Nonce. Due to the avalanche effect inherent to the SHA-256 algorithm, modifying even a single character or digit inside a transaction completely mutates the resulting hexadecimal digest. Because the block's recorded hash no longer matches the recomputed state hash, the validation engine detects the anomaly immediately and returns the exact index of the compromised block.

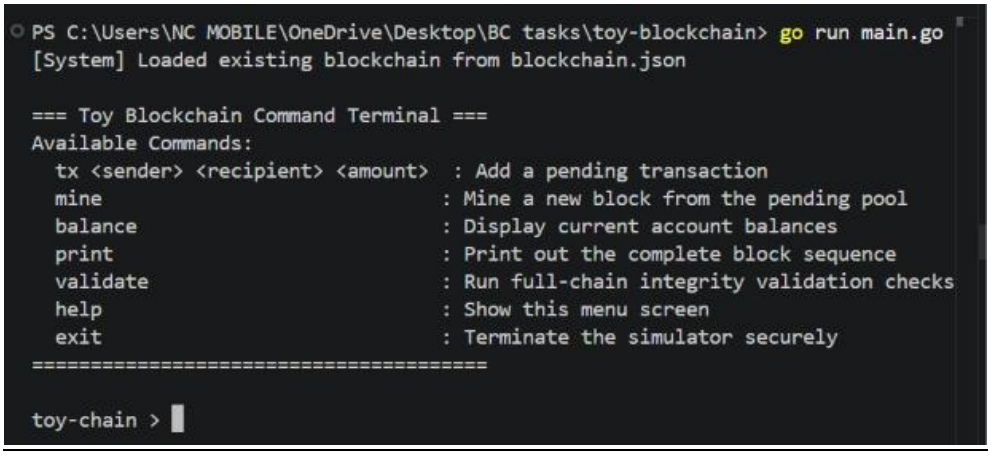

Figure 1Figure 1: Terminal simulation logs demonstrating the validation engine successfully catching historical data modification and hash forgery attempts.

1.2 Difficulty versus Effort Benchmarks

The mining engine was calibrated across progressive cryptographic difficulty targets. Each progressive difficulty level increments the required count of matching leading zero hexadecimal digits (N) mandated for the Proof-of-Work threshold. The empirical performance profile collected on the local hardware architecture is detailed below:

- **Difficulty Target 1:** Nonce Found: 12 | Time Elapsed: 0s
- **Difficulty Target 2:** Nonce Found: 470 | Time Elapsed: 858.8 µs
- **Difficulty Target 3:** Nonce Found: 2,557 | Time Elapsed: 2.58 ms
- **Difficulty Target 4:** Nonce Found: 24,577 | Time Elapsed: 29.54 ms
- **Difficulty Target 5:** Nonce Found: 587,747 | Time Elapsed: 649.27 ms

1.3 Architecture Design Write-Up

Hashing Configuration

We utilize standard SHA-256 hashing via Go's standard library (crypto/sha256). To guarantee strict mathematical determinism across executions, block fields are extracted into an isolated HashInput abstraction layer that orders fields sequentially:

Index -> Timestamp -> Transactions -> PrevHash -> Nonce

The structural Hash field itself is intentionally omitted from this serialization loop to prevent mathematical circular dependencies.

Chain-Wide Integrity Guarantees

The integrity of the ledger depends on sequential back-linked dependencies. Because Block X explicitly embeds the exact string of the previous block's hash inside its structure, changing the historical transactions of Block X-1 rewrites its own hash. This instantly causes a structural mismatch in Block X's PrevHash property. Consequently, to forge a single entry undetected, an attacker must re-calculate every single Proof-of-Work nonce for all succeeding blocks across the system.

2. Discussion Questions

2.1 The Impracticality of Tampering in Production

In our local single-process simulator, tampering is straightforward because the state lives entirely within a localized memory heap or individual JSON file. In a production decentralized blockchain network, tampering becomes practically impossible due to distributed consensus protocols and the longest-chain rule.

Even if an attacker possesses the raw processing power to re-mine subsequent blocks locally on their own isolated node, their modified ledger will be instantly rejected by the network peers. Network nodes enforce strict peer-to-peer verification protocols; they will only accept a chain that matches the majority consensus and represents the maximum cumulative Proof-of-Work difficulty. Overriding this requires a 51% attack, where a malicious entity must control more computing hardware than the remainder of the global network combined—rendering tampering financially and structurally prohibitive.

2.2 Consensus Alternatives: Proof-of-Stake (PoS)

- **Mechanism:** Instead of node operators racing to solve resource-intensive arithmetic loops via processing hardware, blocks are produced by selected validators who commit a quantity of native token capital ("stake") as a collateral deposit.
- **Advantage over PoW:** Massive operational sustainability. It completely eliminates the massive energy consumption, carbon footprints, and specialized hardware demands (ASICs) that characterize conventional Proof-of-Work setups.
- **Drawback over PoW:** Accumulation centralization risk ("the rich get richer"). Because selection probability scales proportionally with the quantity of tokens locked up, wealthy participants accumulate higher validator control and collect the majority of network rewards, threatening decentralized governance.

2.3 Structural Gaps: Toy vs. Production

- **Network Layer Consensus:** The toy is localized to a single process. Production systems use distributed peer-to-peer gossip communication frameworks (like Libp2p) to keep thousands of isolated ledger nodes in sync.

- **Asymmetric Transaction Security:** Our model uses unverified text tags for accounts. Real-world chains mandate public/private key-pair typography (like ECDSA or Ed25519) so every transfer carries a cryptographic digital signature proving structural authorization.
- **Optimized Structural Hashing (Merkle Trees):** We serialize raw transaction lists directly into block data. Production environments build hierarchical Merkle Trees, maintaining a single Merkle Root inside the block header to facilitate rapid, light-weight asset validation without parsing full transaction arrays.

Architectural Sketch: Implementing a Merkle Root

To swap out raw transaction listing hashes for an elegant Merkle Root design, the core block processing loop would be updated as follows:

Plaintext

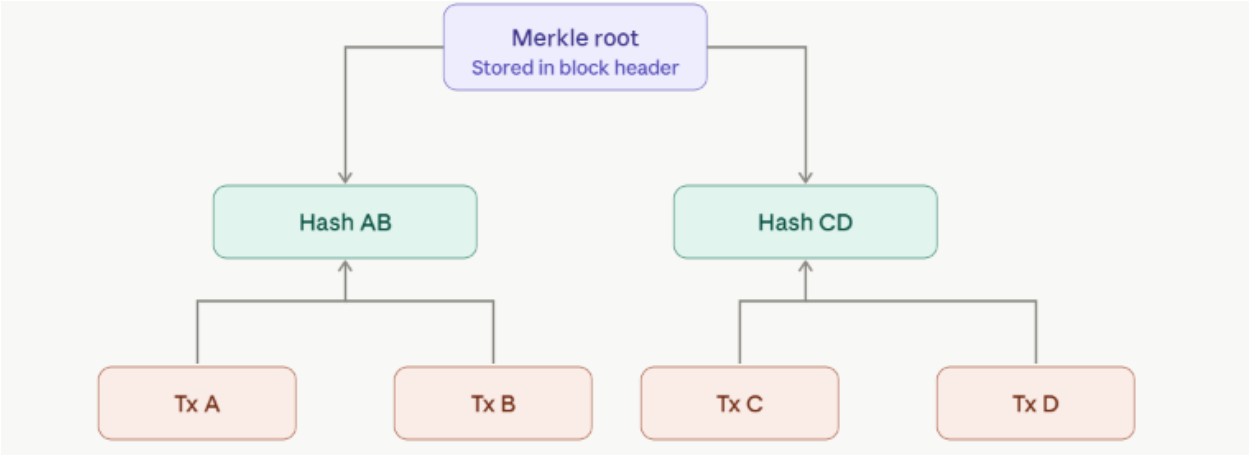

Figure 2Raw transaction hashing

1. Each individual transaction inside a block is hashed once using SHA-256.
2. The system pairs adjacent leaves up sequentially and hashes their string components together.
3. This cryptographic pairing continues up the tree until a single master hash remains—the Merkle Root.
4. The structural HashInput layout will no longer include the variable-length transaction slice; instead, it will hold a single, uniform 32-byte MerkleRoot string. This allows lightweight clients to safely confirm a transaction's existence using a minimal path proof without loading full blocks.